

Contenido semana 5

2. Modelos de Ciclo de Vida del Software

Resultado de aprendizaje: Selecciona el modelo de ciclo de vida del software al inicio de un proyecto de desarrollo, con base en las características del equipo de desarrollo, del cliente, así como de los aspectos vinculados con la problemática por solucionar.

Ciclo de Vida en Cascada

El modelo en cascada (a veces llamado ciclo de vida clásico), sugiere un enfoque sistemático y secuencial para el desarrollo del software, que comienza con la especificación de los requerimientos por parte del cliente (comunicación) y avanza a través de planeación, modelado, construcción y despliegue. En la figura 1, se puede observar estas fases.

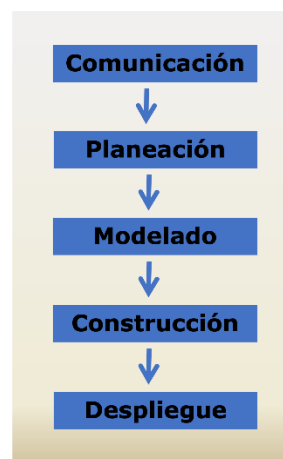


Fig 1. Fases del modelo en cascada

En la figura 2 se desglosan las actividades que se realizan en cada fase del modelo en cascada y enseguida se describen cada una de las actividades:

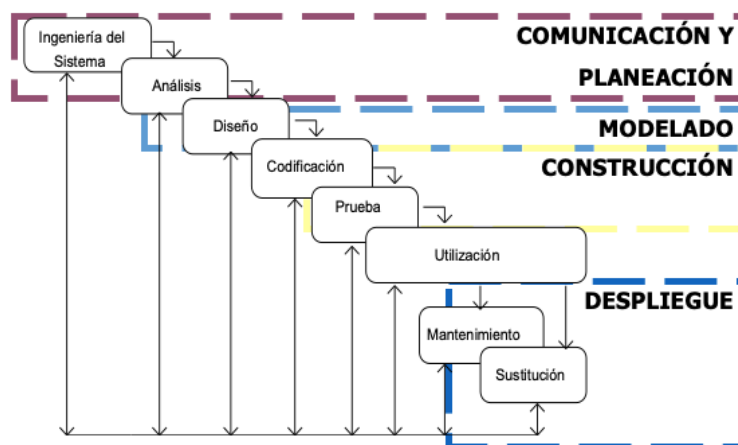


Fig 2. Actividades del modelo en cascada.

Glosario:

Análisis y definición de requerimientos

Este proceso consiste en definir y establecer los servicios, las restricciones y las metas del sistema, esto se hace mediante una comunicación directa con el usuario del sistema, también llamado cliente. Posteriormente se definen a detalle cada uno de estos servicios, y restricciones y sirven como una especificación del sistema.

Diseño del sistema y del software

Este proceso establece una arquitectura general del sistema mediante la asignación de los requerimientos (hardware o software). El diseño del software implica identificar y describir las abstracciones fundamentales del sistema de software y sus relaciones.

Implementación y prueba de unidad

Durante esta etapa, el software se realiza como un conjunto de programas o unidades del programa, cada unidad de programa se debe validar o verificar que cumpla con la especificación.

Integración y prueba de sistema

En esta etapa, una vez probadas las unidades de programa, estos se integran integran y prueban como un sistema completo para asegurarse de que el sistema en su totalidad y como una unidad completa, cumpla con los requerimientos de software. Una vez probado, se "libera" (entrega) el sistema de software al cliente.

Operación y mantenimiento

Esta suele ser la fase más larga del ciclo de vida del sistema de software, en esta etapa el sistema se instala y se coloca a disposición del cliente para su utilización. Esta etapa incluye el mantenimiento en el cual se identifican los errores que no se detectaron en etapas anteriores del ciclo de vida, se mejora la implementación de las unidades del sistema y muchas veces se incrementar los servicios del sistema conforme se descubren nuevos requerimientos.

El modelo fue presentado por primera vez por Royce en 1970. Es un proceso de desarrollo secuencial que fluye como una "cascada" a través de todas las fases de un proyecto (análisis, diseño, desarrollo y pruebas, por ejemplo), y cada fase concluye completamente antes de la siguiente. Este modelo obliga a especificar lo que el sistema hace antes de construir tales componentes, es decir, obliga a definir cómo van a interactuar los componentes antes de construirlos, esto permite al jefe de proyecto seguir y controlar el progreso de manera exacta. Este modelo requiere la generación de una serie de documentos que serán utilizados para la validación y mantenimiento y es adecuado cuando los requerimientos están bien definidos y son estables en forma razonable.

A continuación se listan las ventajas y desventajas del modelo en cascada:

Ventajas

1. La gestión y planificación del proyecto es sencilla y bien definida con un historial comprobado.
2. Cada colaborador del proyecto sabe qué debe hacerse y cuándo, debido a requisitos claramente establecidos.

3. Los desarrolladores pueden detectar errores de diseño durante las etapas de análisis y diseño, lo que les ayuda a evitar escribir código defectuoso durante la fase de implementación.
4. Planificación eficaz del tiempo de la duración del proyecto.
5. El costo total del proyecto se puede estimar con precisión, al igual que el cronograma, una vez que se hayan definido los requisitos.
6. Los desarrolladores que se unan al proyecto en curso pueden integrarse fácilmente porque todo está documentado en el documento de requisitos.
7. Alta Calidad del producto final.

Desventajas

1. Es menos flexible o "ágil" que otras metodologías. Se requieren los requisitos explícitamente desde el principio, es decir, es un método rígido.
2. Los cambios que se producen en etapas posteriores del proceso pueden llevar mucho tiempo, ser dolorosos y costosos.
3. Los proyectos pueden tardar más en entregarse al cliente con este enfoque.
4. Cuando una fase del proceso se retrasa, todas las demás fases se retrasan.
5. Se tiene una primera versión funcional del sistema cuando éste ya está muy avanzado, puede ocasionar problemas.
6. Los clientes a menudo no saben completamente lo que quieren desde el principio, lo que abre la puerta a solicitudes de cambios y nuevas características más adelante en el proceso, cuando son más difíciles de satisfacer.
7. Los clientes no participan en las etapas de diseño e implementación.
8. El cliente no verá resultados hasta el final, con lo que puede impacientarse.
9. Es comparativamente más lento que otros métodos y el costo es mayor también.

Ciclo de Vida Incremental

Muchas veces como ingenieros de software nos vamos a encontrar con situaciones donde los requisitos iniciales de software están bien definidos, pero el enfoque del esfuerzo de desarrollo no sigue un proceso lineal. Quizás también se tenga la necesidad de entregar de manera rápida un conjunto limitado de funcionalidades, ya sea porque le urge al cliente procesos o módulos específicos, y posteriormente, una vez entregados estos se pueden refinar y expandir en entregas posteriores.

En estos casos se elige un modelo de proceso diseñado para producir el software de manera incremental. El modelo incremental es un proceso iterativo de desarrollo de software en el que el producto se diseña, implementa y prueba de forma incremental (se agrega un poco más cada vez) hasta que el producto está terminado. Cada iteración representa una pequeña parte del sistema general e incluye nuevas características y mejoras a las existentes. Se basa en la idea de diseñar una implementación inicial, exponer ésta al comentario del usuario, y luego desarrollarla en sus diversas versiones hasta producir un sistema adecuado.

En este modelo, cada módulo pasa por las fases de requisitos, diseño, implementación y prueba. Cada versión posterior del módulo agrega funciones a la versión anterior. El proceso continúa hasta lograr el sistema completo, esto se puede observar en la figura 1.

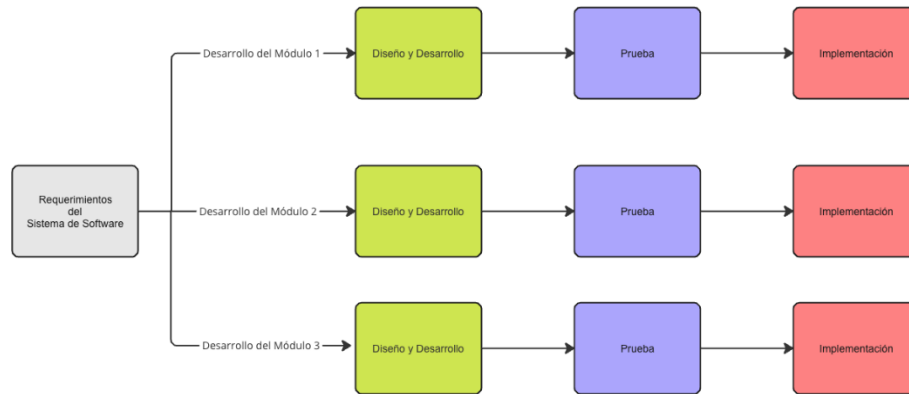


Fig 1. Modelo incremental.

Las fases en el modelo incremental son:

Análisis de requisitos:

En esta primera fase del modelo incremental se confía en la experiencia del equipo de trabajo en el análisis del producto, identificando con esto los requisitos del software. El equipo de análisis de requisitos comprende los requisitos funcionales del sistema.

Diseño y Desarrollo

En esta fase, el diseño de la funcionalidad del sistema y el método de desarrollo se finalizan con éxito. Cuando el software desarrolla una nueva funcionalidad, el modelo incremental utiliza el estilo y la fase de desarrollo.

Pruebas

En esta fase se verifica el desempeño de cada función existente, así como la funcionalidad adicional. En la fase de prueba, se utilizan diversos métodos para probar el comportamiento de cada tarea.

De manera resumida podemos concluir con algunas características de este modelo de ciclo de vida de desarrollo del software:

- Los incrementos diseñados se entregan al cliente y se implementan para usarse en un entorno operacional.
- Los clientes identifican, en un bosquejo, los servicios que proporciona el sistema.
- Identifican cuáles servicios son más importantes y cuáles son menos significativos para ellos.
- Entonces, se define un número de incrementos de entrega, y cada incremento proporciona un subconjunto de la funcionalidad del sistema.
- La asignación de servicios por incrementos depende de la prioridad del servicio.

- Aplica secuencias lineales en forma escalonada a medida que avanza el calendario de actividades.
- Cada secuencia lineal produce "incrementos" de software susceptibles de entregarse de manera parecida a los incrementos producidos en un flujo de proceso evolutivo.

Algunas de las ventajas y desventajas del modelo incremental se lista a continuación:

Ventajas

Comparado con el modelo en cascada, el desarrollo incremental tiene tres beneficios importantes:

1. Menor costo de integrar los requerimientos cambiantes del cliente. El análisis y la documentación que tiene que hacerse de nuevo son mucho menores de lo que el modelo en cascada requeriría.
2. La retroalimentación del cliente es más fácil de obtener ya que se pueden definir módulos e incrementos pequeños. Los clientes pueden comentar las demostraciones del software y darse cuenta de cuánto se ha implementado en lugar de ver el avance mediante documentos del diseño del software.
3. Es más rápida la entrega e implementación de software útil al cliente aunque no se haya incluido y concluido toda la funcionalidad. Los clientes tienen posibilidad de usar y ganar valor del software antes de que éste finalice por completo, algo de lo que no sería posible con un proceso en cascada.

Desventajas

1. Desde un punto de vista de la administración el proceso puede no ser visible, esto es, los administradores necesitan entregas regulares para medir el avance. Si los sistemas se desarrollan rápidamente, resulta poco efectivo en términos de costos producir documentos que reflejen cada versión del sistema.
2. Es importante saber también que la estructura del sistema de software tiende a degradarse conforme se existen nuevos incrementos. Los diferentes cambios en el sistema debido a los incrementos tienden a corromper la estructura del mismo, por lo que la incorporación de más cambios se vuelve cada vez más difícil y costoso.

Ciclo de Vida Evolutivo

El software como muchas cosas evoluciona con el tiempo, los requisitos de negocio y productos cambian conforme se realiza el desarrollo, por lo que no es conveniente seguir una ruta lineal como el método de cascada.

En el mundo del desarrollo de software existe una alta presión por desarrollar rápido y entregar al mercado un producto de software, aunque no esté finalizado con todos los requisitos, por lo que se debe presentar una versión limitada para liberar esa presión competitiva, si el desarrollo se tarda demasiado debido a la metodología utilizada, tal como el modelo en cascada, al salir al mercado el producto de software es probable que la competencia ya haya liberado uno o más productos similares por lo que ya no sería redituable el proyecto. Por esta y otras razones los ingenieros de software necesitan un modelo de proceso que haya sido diseñado de manera explícita para incluir un producto que evolucione con el tiempo.

Los modelos evolutivos son iterativos, los caracteriza la forma en que permiten que los ingenieros de software desarrollen versiones cada vez más completas del software. Dos de los tipos de modelos evolutivos son: modelo de ciclo de vida en espiral y modelo de ciclo de vida basado en prototipos, los cuales se verán a detalle en las siguientes secciones.

Algunas de las ventajas y desventajas del modelo evolutivo se lista a continuación:

Ventajas

1. Los requerimientos del negocio y del producto cambien conforme avanza el desarrollo, lo que hace que no sea realista trazar una trayectoria rectilínea hacia el producto.
2. Los plazos apretados del mercado hacen que sea imposible la terminación de un software perfecto, pero debe lanzarse una versión limitada a fin de aliviar la presión de la competencia o del negocio.
3. Se comprende bien el conjunto de requerimientos o el producto básico, pero los detalles del producto o extensiones del sistema aún están por definirse.

Desventaja

1. Dificultad para estimar el esfuerzo final necesario.
2. Dificultad para medir el progreso o avance del proyecto.
3. No tiene éxito si no hay implicación del usuario, por lo que es obligatoria la participación del cliente.
4. Se corre el riesgo de que el proyecto no tenga un final, es decir, se siga aumentando los requisitos de manera indefinida.

Ciclo de vida por prototipos

El paradigma de construcción de prototipos es adecuado cuando, por ejemplo, un cliente define un conjunto de objetivos generales para el software, pero no identifica los requisitos detallados de los datos de entrada, procesamiento o salida. Por otro lado, el administrador de un proyecto de software quizás esté inseguro de la eficacia de un algoritmo, de la adopción de un sistema de base como un sistema operativo.

Así, un prototipo de un producto de software es una versión preliminar de un sistema de información con fines de demostración o evaluación. El prototipo de requerimientos es la creación de una implementación parcial de un sistema, el cual es construido de una manera rápida tal como sea posible, se entrega a los usuarios, clientes o representantes de ellos, posibilitando que experimenten con él, posteriormente ellos proveen la retroalimentación sobre lo que a ellos les gustó y no les gustó del prototipo, los ingenieros de software entonces capturan en la documentación actual de la especificación de requerimientos la información entregada por los usuarios para el desarrollo del sistema real.

El prototipo puede ser usado como parte de la fase de requerimientos (determinar requerimientos) o justo antes de la fase de requerimientos (como predecesor de requerimientos). En otro caso, el prototipo puede servir su papel inmediatamente antes de algún o todo el desarrollo incremental en modelos incremental o evolutivo. Se usa frecuentemente, porque la especificación de requerimientos para

sistemas complejos tiende a ser relativamente dificultoso de llevar a cabo. Muchos usuarios y clientes encuentran que es mucho más fácil proveer retroalimentación convenientemente usando el prototipo en lugar de leer una especificación de requerimientos potencialmente ambigua y extensa.

En la figura 1 se puede observar las diferentes fases del modelo basado en prototipos y posteriormente se describe a detalle cada una de ellas.

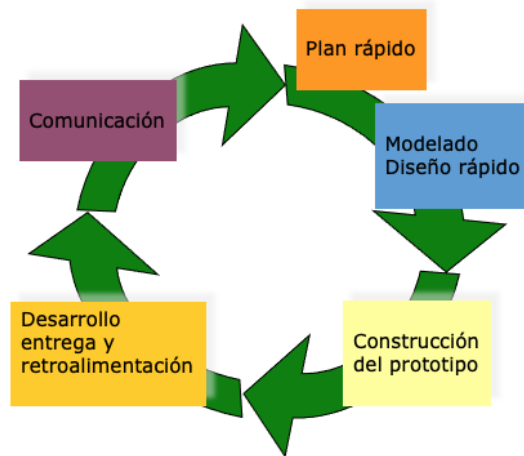


Fig 1. Modelo basado en prototipos

- El paradigma de prototipos comienza con la comunicación con el cliente, recopilar los requisitos, aunque el cliente no sabe mucho sobre los requisitos, pero los requisitos principales están definidos en detalle.
- En el plan rápido se planea la construcción de un prototipo y se presenta el modelado.
- En el modelado de diseño rápido se centra en una representación de aquellos aspectos del software que serán visibles para el cliente o el usuario final.
- Se construye el prototipo, se muestran algunos requisitos básicos y la interfaz de usuario que estaría disponible.
- El prototipo se entrega y es evaluado por los participantes, que dan retroalimentación para mejorar los requerimientos. Esta retroalimentación se utiliza para mejorar aún más el sistema y se realizan posibles cambios en el prototipo.
- La iteración ocurre a medida que el prototipo es afinado para satisfacer las necesidades.
- Cuando se reciben comentarios de los usuarios finales y clientes, el prototipo se mejora en función de los comentarios. Si el cliente no está satisfecho con el prototipo, se crea un nuevo prototipo y este proceso continúa hasta que el cliente obtiene el prototipo según sus necesidades.

A continuación se listan las ventajas y desventajas del ciclo de vida por prototipos:

Ventajas

1. Este modelo es adecuado para crear aplicaciones cuyo prototipo sea muy sencillo y que siempre incluya la interacción humano-computadora, es decir, para el desarrollo de sistemas inteligentes,

de inteligencia artificial o aprendizaje máquina, no son recomendables, a menos que sean aplicaciones de software donde el usuario tenga una alta interacción.

2. Es adecuado cuando se conoce sólo el objetivo general del software, pero no sabemos nada en detalle sobre las entradas, el procesamiento y la salida. En esta situación, se convierte en un modelo basado en prototipo.
3. Cuando el desarrollador de software no está muy seguro de la capacidad de un algoritmo o de su adaptabilidad a un sistema operativo, entonces, en esta situación.

Desventajas

1. Cuando la primera versión del prototipo ya está lista, el cliente a menudo desea realizar pequeñas modificaciones en lugar de reconstruir el sistema. Si se rediseña el sistema se mantendrá más calidad en el mismo.
2. Se pueden comprometer muchos otros requisitos en la primera versión del modelo prototip, el usuario puede pedir más para una siguiente versión del prototipo.
3. Un desarrollador de software puede hacer concesiones (algo que quizás no fue solicitado por el cliente) en su implementación, sólo para que el modelo prototipo esté en funcionamiento rápidamente. Esto puede ocasionar problemas futuros con el cliente.

Ciclo de vida en espiral

Boehm (1988) propuso un marco del proceso de software dirigido por el riesgo (el modelo en espiral). En este modelo, el proceso de software se representa como una espiral, y no como una secuencia de actividades con cierto retroceso de una actividad a otra. Cada ciclo en la espiral representa una fase del proceso de software. Por ende, el ciclo más interno puede relacionarse con la factibilidad del sistema, el siguiente ciclo con la definición de requerimientos, el ciclo que sigue con el diseño del sistema, etcétera. El modelo en espiral combina el evitar el cambio con la tolerancia al cambio. Lo anterior supone que los cambios son resultado de riesgos del proyecto e incluye actividades de gestión de riesgos explícitas para reducir tales riesgos.

Es un enfoque realista para el desarrollo de sistemas y de software a gran escala. El software evoluciona a medida que el proceso avanza, el desarrollador y el cliente comprenden y reaccionan mejor ante los riesgos de cada nivel de evolución. Toma las ventajas del modelo de desarrollo en cascada y el de prototipos añadiéndole el concepto de análisis de riesgo

Tiene dos características principales:

- Tiene un enfoque cíclico para el crecimiento incremental del grado de definición de un sistema y su implementación, mientras que disminuye su grado de riesgo. Este reconocimiento explícito del riesgo es la diferencia principal de este modelo y los demás.
- Es un conjunto de puntos de referencia de anclaje puntual para asegurar el compromiso del participante con soluciones factibles y mutuamente satisfactorias.

En la figura 1 se puede observar las diferentes fases del modelo en espiral y posteriormente se describe a detalle cada una de ellas.

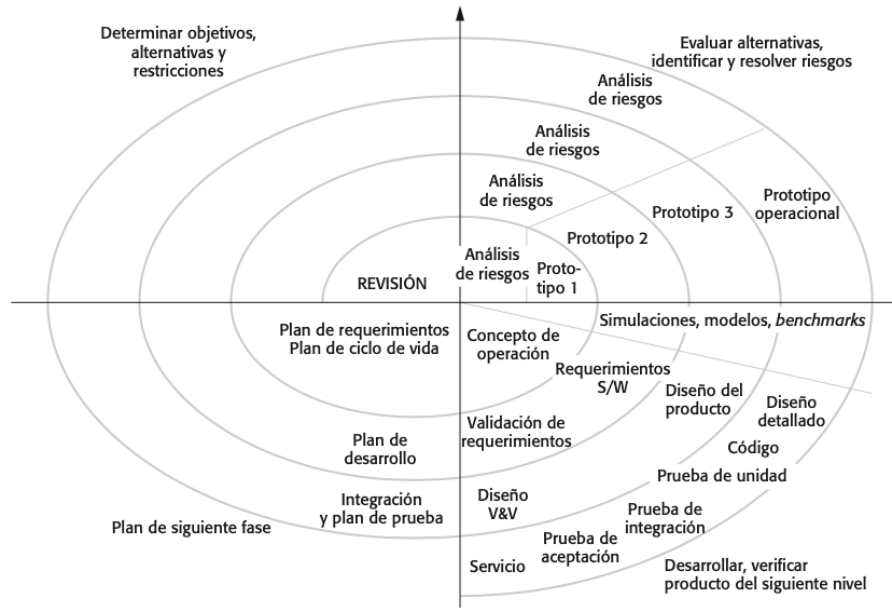


Fig 1. Modelo en espiral (sommerville)

Se definen cuatro actividades básicas:

Establecimiento de objetivos

Se definen objetivos específicos para dicha fase del proyecto. Se identifican restricciones en el proceso y el producto, y se traza un plan de gestión detallado. Se identifican los riesgos del proyecto. Pueden planearse estrategias alternativas, según sean los riesgos.

Valoración y reducción del riesgo

En cada uno de los riesgos identificados del proyecto, se realiza un análisis minucioso. Se dan acciones para reducir el riesgo. Por ejemplo, si existe un riesgo de que los requerimientos sean inadecuados, puede desarrollarse un sistema prototipo.

Desarrollo y validación

Después de una evaluación del riesgo, se elige un modelo de desarrollo para el sistema. Por ejemplo, la creación de prototipos desechables sería el mejor enfoque de desarrollo, si predominan los riesgos en la interfaz del usuario.

Planeación

El proyecto se revisa y se toma una decisión sobre si hay que continuar con otro ciclo de la espiral. Si se opta por continuar, se trazan los planes para la siguiente fase del proyecto.

Algunas ventajas y desventajas del ciclo en espiral son:

Ventajas

1. Se presta atención a las opciones que permiten la reutilización de software existente.

2. Se centra en la eliminación de errores y alternativas poco atractivas.
3. No establece una diferenciación entre desarrollo de software y mantenimiento del sistema.
4. Proporciona un marco estable para desarrollos integrados hardware-software.

Desventajas

1. Dificultad de convencer sobre su enfoque evolutivo controlable.
2. Demanda mucha experiencia en la evaluación del riesgo y se basa en ella para el éxito.